

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregations for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- SCD Type 2 implementation for order summary tracking
- Incremental processing logic triggered by customer updates
- Aggregation of customer spending metrics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Specific scheduling or orchestration mechanisms
- Data retention or archival policies
- Detailed monitoring and alerting configurations

Assumptions

1. CSV files in the source S3 paths follow consistent schema and naming conventions
2. The AWS Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists
3. S3 bucket `adif-sdlc` and required paths have appropriate permissions configured
4. Apache Hudi libraries are available in the AWS Glue environment
5. Date fields in source data are in a parseable format (specific format not provided in BRD)
6. "Null" string and NULL values are the only data quality issues to address

7. Customer updates are identifiable through change detection logic (mechanism not specified in BRD)

8. TotalAmount calculation is derived from $\text{PricePerUnit} \times \text{Qty}$

9. Athena will be used for querying the Glue Catalog tables

10. CloudWatch will be used for job monitoring and logging

•

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3

- Description:

The system shall read customer data in CSV format from the specified S3 location and make it available for downstream processing.

- Business Objective:

Enable the ETL pipeline to access current customer information for order processing and analytics.

- Inputs:

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/

- Format: CSV

- Schema:

- CustId: (datatype not specified)

- Name: (datatype not specified)

- EmailId: (datatype not specified)

- Region: (datatype not specified)

- Processing Rules / Functional Steps:

1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/customerdata/

2. Read all CSV files matching the customer data pattern

3. Parse CSV content according to the defined schema

4. Load data into a DataFrame or equivalent processing structure

- Outputs:

- In-memory customer dataset ready for data quality processing

- Preconditions:

- S3 path exists and contains at least one CSV file

- AWS Glue job has read permissions on the S3 path
- Postconditions:
- Customer data is loaded and accessible for subsequent processing steps
- Error / Validation Conditions:
- If S3 path does not exist, job should fail with clear error message
- If CSV files cannot be parsed, job should fail with schema mismatch error
- If no files are found, job should fail or log warning based on configuration
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order data in CSV format from the specified S3 location and make it available for downstream processing.

- Business Objective:

Enable the ETL pipeline to access transactional order information for customer order analysis.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified)
- ItemName: (datatype not specified)
- PricePerUnit: (datatype not specified)
- Qty: (datatype not specified)
- Date: (datatype not specified)
- CustId: (datatype not specified)
- Processing Rules / Functional Steps:
- 1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files matching the order data pattern
- 3. Parse CSV content according to the defined schema
- 4. Load data into a DataFrame or equivalent processing structure
- Outputs:

- In-memory order dataset ready for data quality processing
- Preconditions:
 - S3 path exists and contains at least one CSV file
 - AWS Glue job has read permissions on the S3 path
- Postconditions:
 - Order data is loaded and accessible for subsequent processing steps
- Error / Validation Conditions:
 - If S3 path does not exist, job should fail with clear error message
 - If CSV files cannot be parsed, job should fail with schema mismatch error
 - If no files are found, job should fail or log warning based on configuration
-

FR-CLEAN-001: Remove Null Values and Duplicates from Customer Data

- Title: Data Quality Processing for Customer Dataset
- Description:

The system shall remove records containing "Null" string values, NULL values, and duplicate records from the customer dataset.

- Business Objective:

Ensure data quality and consistency by eliminating invalid and redundant customer records before catalog registration and downstream processing.

- Inputs:
 - In-memory customer dataset from FR-INGEST-001
- Processing Rules / Functional Steps:
 1. Identify and remove rows where any field contains the string "Null"
 2. Identify and remove rows where any field contains NULL value
 3. Identify duplicate records across all customer fields
 4. Remove duplicate records, retaining only one instance of each unique customer record
- Outputs:
 - Cleaned customer dataset with no null values or duplicates
- Preconditions:
 - Customer data has been successfully loaded (FR-INGEST-001 completed)

- Postconditions:
- Customer dataset contains only valid, unique records
- Record count may be reduced from original input
- Error / Validation Conditions:
- If all records are removed due to null values, job should fail or log critical warning
- Deduplication logic should be deterministic and repeatable
-

FR-CLEAN-002: Remove Null Values and Duplicates from Order Data

- Title: Data Quality Processing for Order Dataset

- Description:

The system shall remove records containing "Null" string values, NULL values, and duplicate records from the order dataset.

- Business Objective:

Ensure data quality and consistency by eliminating invalid and redundant order records before catalog registration and downstream processing.

- Inputs:
- In-memory order dataset from FR-INGEST-002
- Processing Rules / Functional Steps:
 1. Identify and remove rows where any field contains the string "Null"
 2. Identify and remove rows where any field contains NULL value
 3. Identify duplicate records across all order fields
 4. Remove duplicate records, retaining only one instance of each unique order record
- Outputs:
- Cleaned order dataset with no null values or duplicates
- Preconditions:
- Order data has been successfully loaded (FR-INGEST-002 completed)
- Postconditions:
- Order dataset contains only valid, unique records
- Record count may be reduced from original input
- Error / Validation Conditions:
- If all records are removed due to null values, job should fail or log critical warning

- Deduplication logic should be deterministic and repeatable
-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Data
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Make customer data discoverable and queryable through AWS analytics services.

- Inputs:
 - Cleaned customer dataset from FR-CLEAN-001
 - Target database: gen_ai_poc_databrickscoe
 - Target table name: sdhc_wizard_customer
- Processing Rules / Functional Steps:
 1. Write cleaned customer data to S3 in a queryable format
 2. Register table metadata in Glue Data Catalog
 3. Set database to gen_ai_poc_databrickscoe
 4. Set table name to sdhc_wizard_customer
 5. Define schema matching the customer data structure
- Outputs:
 - Glue Catalog table: gen_ai_poc_databrickscoe.sdhc_wizard_customer
 - Customer data persisted in S3
- Preconditions:
 - Customer data cleaning completed (FR-CLEAN-001)
 - Database gen_ai_poc_databrickscoe exists in Glue Data Catalog
 - AWS Glue job has permissions to create/update catalog tables
- Postconditions:
 - Table is queryable via Athena
 - Table metadata is visible in Glue Data Catalog
- Error / Validation Conditions:
 - If database does not exist, job should fail with clear error

- If table creation fails, job should fail with catalog error details
-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Data
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Make order data discoverable and queryable through AWS analytics services.

- Inputs:
 - Cleaned order dataset from FR-CLEAN-002
 - Target database: gen_ai_poc_databricksco
 - Target table name: sdlc_wizard_order
- Processing Rules / Functional Steps:
 1. Write cleaned order data to S3 in a queryable format
 2. Register table metadata in Glue Data Catalog
 3. Set database to gen_ai_poc_databricksco
 4. Set table name to sdlc_wizard_order
 5. Define schema matching the order data structure
- Outputs:
 - Glue Catalog table: gen_ai_poc_databricksco.sdlc_wizard_order
 - Order data persisted in S3
- Preconditions:
 - Order data cleaning completed (FR-CLEAN-002)
 - Database gen_ai_poc_databricksco exists in Glue Data Catalog
 - AWS Glue job has permissions to create/update catalog tables
- Postconditions:
 - Table is queryable via Athena
 - Table metadata is visible in Glue Data Catalog
- Error / Validation Conditions:
 - If database does not exist, job should fail with clear error

- If table creation fails, job should fail with catalog error details
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Maintain Order Summary with Slowly Changing Dimension Type 2 Logic
- Description:

The system shall join customer and order data, and maintain a historical record of order summaries using SCD Type 2 methodology with Apache Hudi, tracking changes over time with active/inactive flags and effective date ranges.

- Business Objective:

Provide historical tracking of order summaries to support time-based analysis and auditing of customer order changes.

- Inputs:
- Cleaned customer dataset from FR-CLEAN-001
- Cleaned order dataset from FR-CLEAN-002
- Existing ordersummary table (if present) from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Processing Rules / Functional Steps:

1. Join customer and order datasets on CustId field
2. For each joined record, check if it exists in the current ordersummary table
3. If record is new or has changed:
 - Close existing active row by setting IsActive = false, EndDate = current timestamp
 - Insert new row with IsActive = true, StartDate = current timestamp, EndDate = null
4. If record is unchanged, no action required
5. Maintain the following SCD2 control fields:
 - IsActive: Boolean flag indicating current active record
 - StartDate: Timestamp when record became active
 - EndDate: Timestamp when record became inactive (null for active records)
 - OpTs: Operation timestamp for tracking
6. Write results using Apache Hudi format to S3

- Outputs:
- Updated ordersummary table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary
- Historical records maintained with SCD2 attributes

- Preconditions:
- Customer and order data have been cleaned and cataloged
- S3 path s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ is accessible
- Apache Hudi libraries are available in Glue environment
- Postconditions:
- All current records have IsActive = true
- Historical records have IsActive = false with appropriate date ranges
- No gaps in historical tracking
- Table is queryable via Athena with time-travel capabilities
- Error / Validation Conditions:
- If join produces no results, log warning but do not fail
- If Hudi write fails, job should fail with detailed error
- Ensure no duplicate active records exist for the same business key
-

FR-INCR-001: Process Incremental Customer Updates

- Title: Trigger SCD2 Updates Based on Customer Changes
- Description:

The system shall detect changes in customer data and trigger corresponding updates to the ordersummary table using SCD Type 2 logic.

- Business Objective:

Ensure order summary records reflect the most current customer information while maintaining historical accuracy.

- Inputs:
- New or updated customer records from s3://adif-sdlc/sdlc_wizard/customerdata/
- Existing customer records from gen_ai_poc_databrickscoe.sdlc_wizard_customer
- Current ordersummary table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
- 1. Identify customer records that are new or have changed since last processing
- 2. For each changed customer record:
- Retrieve all related order summary records
- Apply SCD2 update logic as defined in FR-SCD2-001
- Close existing active rows

- Insert new active rows with updated customer information
- 3. Update OpTs timestamp for all affected records
- 4. Maintain IsActive, StartDate, EndDate fields according to SCD2 rules
- Outputs:
 - Updated ordersummary table with incremental changes applied
 - Audit trail of customer-driven changes in historical records
- Preconditions:
 - Customer data has been ingested and cleaned
 - Ordersummary table exists with SCD2 structure
 - Change detection mechanism is operational
- Postconditions:
 - All order summaries reflect current customer information for active records
 - Historical records preserve previous customer states
 - Incremental processing is complete and consistent
- Error / Validation Conditions:
 - If no customer changes detected, processing should complete successfully with no updates
 - If change detection fails, job should fail with clear error
 - Ensure referential integrity between customer and order summary records
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Aggregated Customer Spending Analytics
- Description:

The system shall aggregate order data by customer to calculate total spending amounts and produce an analytics table for business intelligence purposes.

- Business Objective:

Provide summarized customer spending metrics to support business analysis and reporting.

- Inputs:
 - Cleaned customer dataset from FR-CLEAN-001
 - Cleaned order dataset from FR-CLEAN-002
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId

2. Calculate TotalAmount for each order as $\text{PricePerUnit} \times \text{Qty}$
3. Group results by customer Name and Date
4. Sum TotalAmount for each customer-date combination
5. Produce output with columns: Name, TotalAmount, Date
6. Write results to S3 location `s3://adif-sdlc/analytics/customeraggregatespend/`

- Outputs:

- Analytics table at `s3://adif-sdlc/analytics/customeraggregatespend/`
- Glue Catalog table: `gen_ai_poc_databrickscoe.customeraggregatespend`
- Schema: Name, TotalAmount, Date

- Preconditions:

- Customer and order data have been cleaned
- S3 path `s3://adif-sdlc/analytics/customeraggregatespend/` is accessible

- Postconditions:

- Aggregated spending data is available for querying
- Table is registered in Glue Data Catalog
- Data is optimized for analytical queries

- Error / Validation Conditions:

- If join produces no results, create empty table with schema
- If TotalAmount calculation fails due to invalid numeric values, log error and skip record
- Ensure no duplicate aggregations for same customer-date combination

-

FR-CATALOG-003: Register Order Summary Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Summary

- Description:

The system shall register the ordersummary Hudi table in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Make SCD2 order summary data discoverable and queryable through AWS analytics services.

- Inputs:

- Ordersummary Hudi table from `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`
- Target database: `gen_ai_poc_databrickscoe`

- Target table name: ordersummary
- Processing Rules / Functional Steps:
 1. Register Hudi table metadata in Glue Data Catalog
 2. Set database to gen_ai_poc_databricksco
 3. Set table name to ordersummary
 4. Configure Hudi-specific table properties for Athena compatibility
 5. Define schema including SCD2 control fields (IsActive, StartDate, EndDate, OpTs)
- Outputs:
 - Glue Catalog table: gen_ai_poc_databricksco.ordersummary
 - Athena-queryable Hudi table with time-travel capabilities
- Preconditions:
 - Ordersummary Hudi table has been created (FR-SCD2-001)
 - Database gen_ai_poc_databricksco exists in Glue Data Catalog
- Postconditions:
 - Table is queryable via Athena with Hudi support
 - SCD2 historical queries are supported
- Error / Validation Conditions:
 - If Hudi table registration fails, job should fail with catalog error
 - Verify Hudi table properties are correctly set for Athena access
-

FR-CATALOG-004: Register Customer Aggregate Spend Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Aggregate Spend
- Description:

The system shall register the customeraggregatespend analytics table in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Make aggregated customer spending data discoverable and queryable through AWS analytics services.

- Inputs:
 - Customer aggregate spend data from s3://adif-sdlc/analytics/customeraggregatespend/
 - Target database: gen_ai_poc_databricksco
 - Target table name: customeraggregatespend

- Processing Rules / Functional Steps:

1. Write aggregated data to S3 in a queryable format
2. Register table metadata in Glue Data Catalog
3. Set database to gen_ai_poc_databricksco
4. Set table name to customeraggregatespend
5. Define schema: Name, TotalAmount, Date

- Outputs:

- Glue Catalog table: gen_ai_poc_databricksco.customeraggregatespend
- Analytics data persisted in S3

- Preconditions:

- Customer aggregate spend data has been generated (FR-AGG-001)
- Database gen_ai_poc_databricksco exists in Glue Data Catalog

- Postconditions:

- Table is queryable via Athena
- Aggregated data is accessible for BI tools and reporting

- Error / Validation Conditions:

- If table creation fails, job should fail with catalog error details
- Verify schema matches aggregated data structure

-

3. Non-Functional Considerations

Monitoring and Observability

- AWS CloudWatch shall be used for job execution monitoring and logging
- Job metrics, errors, and execution duration should be logged to CloudWatch

Data Governance (Optional)

- AWS Lake Formation may be used for fine-grained access control and data governance
- If implemented, Lake Formation policies should align with organizational data security requirements

Query Performance

- Athena queries against Glue Catalog tables should be optimized through appropriate file formats and partitioning strategies

Storage Architecture

- All data storage shall use Amazon S3 as the primary storage layer
- Data organization follows a three-tier structure: raw (source), curated (processed), and analytics (aggregated)
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Key Traceability Points	
-----	-----	-----	
FR-INGEST-001	Source Input (1), Schemas (2)	Customer CSV from s3://adif-sdlc/sdlc_wizard/customerdata/	
FR-INGEST-002	Source Input (1), Schemas (2)	Order CSV from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-CLEAN-001	Data Cleaning (3)	Remove "Null" and NULL from customer data	
FR-CLEAN-002	Data Cleaning (3)	Remove "Null" and NULL from order data	
FR-CATALOG-001	Glue Catalog Tables (4)	sdlc_wizard_customer in gen_ai_poc_databrickscoe	
FR-CATALOG-002	Glue Catalog Tables (4)	sdlc_wizard_order in gen_ai_poc_databrickscoe	
FR-SCD2-001	SCD Type 2 (5)	ordersummary with Hudi at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	
FR-INCR-001	Incremental Logic (6)	Customer updates trigger SCD2 in ordersummary	
FR-AGG-001	Aggregation Table (7)	customeraggregatespend at s3://adif-sdlc/analytics/customeraggregatespend/	
FR-CATALOG-003	Glue Catalog Tables (4)	ordersummary in gen_ai_poc_databrickscoe	
FR-CATALOG-004	Glue Catalog Tables (4)	customeraggregatespend in gen_ai_poc_databrickscoe	

- Cross-Requirement Dependencies:
 - FR-CLEAN-001 depends on FR-INGEST-001
 - FR-CLEAN-002 depends on FR-INGEST-002
 - FR-CATALOG-001 depends on FR-CLEAN-001
 - FR-CATALOG-002 depends on FR-CLEAN-002
 - FR-SCD2-001 depends on FR-CLEAN-001 and FR-CLEAN-002
 - FR-INCR-001 depends on FR-SCD2-001
 - FR-AGG-001 depends on FR-CLEAN-001 and FR-CLEAN-002
 - FR-CATALOG-003 depends on FR-SCD2-001
 - FR-CATALOG-004 depends on FR-AGG-001
- Original Source Preservation:

All functional requirements maintain exact references to the original S3 paths, database names, and table names as specified in the BRD. No alternative sources or locations have been substituted.